University of Colorado Denver

Department of Computer Science & Engineering

CSCI/CSCY 4407: Security & Cryptography
Lab 4 Assignment: Cryptographic Hash Functions Attacks
Instructor: Dr. Victor Kebande
Teaching Assistant: Celest Kester
Spring 2026

Assignment Instructions

This is a **group assignment**. Each group is required to complete and submit its own original work. You may discuss general security and cryptography concepts with classmates; however, **you are not permitted to share, reuse, or copy** solutions, calculations, code, scripts, screenshots, written explanations, forged tags, recovered passwords, collision samples, experimental results, or final answers across groups.

For this lab, some tasks require you to **generate your own messages, digests, and experimental data**, while others require you to use **group-specific hash artifacts** provided for controlled attack demonstrations.

- **Tasks 1, 2, 3, 5, 6, and 7:** You must generate your own messages, random inputs, truncated digests, password lists, salts, and benchmarking datasets as instructed in the lab manual. All experimental data must be produced by your group.
- **Task 4 (Length-Extension Vulnerability):** Each group has been assigned a **unique server artifact package** generated using different secret keys and initial message parameters. These files simulate a controlled insecure MAC construction based on a Merkle–Damgård hash.

Because each group receives different cryptographic artifacts (e.g., initial digests, message formats, and hidden key lengths), **all forged outputs, recovered structures, collision**

points, and intermediate values will differ across groups. This is intentional and forms part of the assessment design.

It is your responsibility to ensure that you download and use the correct files assigned to your group. The group-specific files are available on Canvas under:

Modules → Lab 4: Cryptographic Hash Functions Attacks

Each group should download only their assigned file:

`HashServer_Group_XX_Student.zip`

Using another group's artifact package will result in incorrect outputs and may be considered an academic integrity violation. Failure to follow these instructions may lead to grade penalties.

Important: All attack demonstrations in this laboratory are conducted in a controlled academic environment to understand why insecure constructions fail. These techniques must not be applied outside the laboratory setting.

Academic Integrity and Collaboration

- **Allowed:** Discussing lecture material and cryptographic concepts at a high level.
- **Not allowed:** Sharing code, decrypted messages, keys, screenshots, or report content with other groups.
- **Not allowed:** Copying solutions from the internet, generative AI tools, previous students, or public repositories.
- **Submission:** Only **one** group member should submit the final report on Canvas on behalf of the group.

1 Introduction

A cryptographic hash function is a deterministic algorithm that maps data of arbitrary length to a fixed-length output called a *digest* or *message fingerprint*:

$$H : \{0,1\}^* \to \{0,1\}^n.$$

Hash functions are widely used for **integrity verification**, **password storage**, **digital signatures**, and more. Because hash functions are *many-to-one*, collisions *must exist*, but a cryptographic hash is designed so that collisions are **computationally infeasible to find** in practice.

This laboratory is aligned with the course lecture on cryptographic hash functions (properties, SHA-1 padding, Merkle–Damgård construction, collision resistance, birthday attacks, and real-world applications). You will experimentally validate core properties, simulate attacks in controlled settings, and connect theory to real security engineering decisions.

2 Security Perspective

The key security properties typically associated with cryptographic hash functions include:

- **Preimage resistance (one-wayness):** Given y, it should be infeasible to find x such that $H(x) = y$.
- **Second-preimage resistance:** Given x, it should be infeasible to find $x' \neq x$ such that $H(x) = H(x')$.
- **Collision resistance:** It should be infeasible to find *any* pair $x_1 \neq x_2$ such that $H(x_1) = H(x_2)$.

Additionally, the **birthday phenomenon** explains why collisions become likely after about $2^{n/2}$ random trials when the output is n bits, motivating digest sizes such as 256 bits for long-term collision resistance.

3 Objectives

By the end of this lab, you should be able to:

- Experimentally demonstrate the **avalanche effect** and interpret what it means for diffusion.
- Simulate the **birthday paradox** and empirically estimate collision points for a small digest size.
- Implement a **toy Merkle–Damgård** construction using a simple compression function.
- Explain why naive constructions like `SHA1(key || message)` are dangerous and demonstrate the concept of **length-extension** in a controlled lab setting.
- Evaluate password hashing practices by comparing unsalted hashes vs salted slow hashes.
- Use hashes for file integrity checks and reason about trust assumptions.
- Benchmark multiple hash functions and interpret performance differences.

4 Overview of Lab Tasks

Task	Title	Primary Learning Objective

Task 1	Avalanche Effect Experiment	Demonstrate diffusion and unpredictability by measuring bit differences between nearly identical inputs.
Task 2	Birthday Collision Simulation	Experimentally validate the birthday paradox and estimate collision effort for truncated hashes.
Task 3	Toy Merkle–Damgård Construction	Understand iterative compression and chaining structure in hash design.
Task 4	Length-Extension Vulnerability	Demonstrate why naive constructions like $H(k\|\|m)$ are insecure and explain why HMAC prevents this attack.
Task 5	Password Hashing Analysis	Compare unsalted hashing, salted hashing, and slow KDFs (PBKDF2). Analyze dictionary attack feasibility.
Task 6	File Integrity Verification	Use cryptographic hashes for integrity checking and reason about trust assumptions.
Task 7	Performance Benchmarking	Measure and interpret execution time and throughput of SHA-1, SHA-256, and SHA-512.

5 Lab Requirements

5.1 Technical Environment

You must use a Linux-based VM (Kali Linux or Ubuntu recommended) with:

- Python 3.x
- Standard CLI tools: `sha1sum`, `sha256sum`, `xxd`, `time`, `strings`
- Python libraries: `hashlib` (built-in)

5.2 Evidence Requirements (Applies to Every Task)

Each task must include:

- Terminal screenshots showing execution.
- Clear inputs and outputs (hash digests).
- Brief explanation of what the result demonstrates.
- Reproducibility notes: commands used, parameters chosen, file sizes, and any random seeds (if used).

5.3 Important Notes

- **Do not attempt real SHA-256 collisions.** Collision tasks are performed using *small simulated hash spaces* (e.g., 16-bit or 24-bit truncated digests).
- Attack tasks in this lab are **controlled demonstrations** to learn why insecure designs fail.

6 Lab Tasks

6.1 Task 1: Avalanche Effect Experiment

Objective: Demonstrate that small input changes produce large, unpredictable output changes.
Background: A key behavioral expectation in cryptographic hashes is the avalanche effect: flipping one bit in the input should flip roughly half the bits in the digest (statistically, not perfectly).

Step-by-Step Instructions

Step 1: Create two similar messages. In a working directory:

```
mkdir -p lab4_hash && cd lab4_hash
echo -n "Hello world" > m1.txt
echo -n "Hello worle" > m2.txt
ls -l m1.txt m2.txt
```

A sample process for building the task in the directory is shown in Figure 1

Figure 1: Avalanche effect file build

Step 2: Compute hashes using multiple algorithms. Compute SHA-1 and SHA-256:

```
sha1sum m1.txt m2.txt
sha256sum m1.txt m2.txt
```

Step 3: Compute bit-difference between the two digests (SHA-256). Write a Python script `avalanche.py` that:

- reads both messages,
- computes SHA-256 digests in hex,
- converts hex to binary,
- counts how many bits differ,
- outputs: #bits different and percentage.

Step 4: Repeat with different message pairs. Create **at least 5** message pairs where only **one character** differs (or one bit differs if you generate binary files). Record results in a table.
When comparing two messages that differ by only one character, the SHA-256 digests should be completely different. The avalanche script should report approximately 120–140 bits different out of 256, corresponding to about 50% difference. This demonstrates the avalanche effect, where a small change in input produces a large and unpredictable change in the hash output.

6.1.1 Question (Answer Required)

After completing the five avalanche trials, compute the **average** number of flipped bits and the **average percentage** of flipped bits across your trials. Then answer the following:

> **Question:** Based on your results, does SHA-256 demonstrate the avalanche effect? In your answer, (i) cite your computed averages, (ii) explain why "about 50% of bits changing" is expected for a cryptographic hash, and (iii) discuss why repeating multiple trials is necessary rather than relying on a single message pair.

What to Submit

- Screenshots of `sha1sum` and `sha256sum` outputs.
- `avalanche.py` script.
- A results table with 5 trials including: inputs, digests (optional excerpt), bits changed, and percentage changed.
- 1–2 paragraphs: explain avalanche and why it supports unpredictability.

6.2 Task 2: Birthday Collision Simulation (Truncated Hashes)

Objective: Demonstrate the birthday paradox and estimate collision effort.
Background: For an n-bit hash output, collisions become likely after about $2^{n/2}$ random inputs. Since real hashes are large, we simulate by truncating.

Why We Truncate the Hash Output.

In this experiment, we intentionally reduce the size of the SHA-256 output in order to observe collisions within a reasonable amount of time. A full SHA-256 digest is 256 bits long, which makes finding collisions computationally infeasible in practice. To make the birthday phenomenon experimentally observable in a laboratory setting, we define a truncated hash function $H_t(x)$ that keeps only the first t bits of the SHA-256 output (where $t \in \{16, 20, 24\}$) and discards the remaining bits. This effectively shrinks the "fingerprint" size, making collisions much more likely and allowing students to empirically study the birthday paradox. This truncation is performed solely for experimental purposes and does not reflect secure real-world hash usage.

Step-by-Step Instructions

Step 1: Choose a small digest size. Use truncated SHA-256 (e.g., first 16 bits or 24 bits). Define:

$$H_t(x) = \text{first } t \text{ bits of SHA-256}(x)$$

where $t \in \{16, 20, 24\}$.

Step 2: Write a collision-finding script. Create `birthday.py` that:

- generates random inputs,
- computes $H_t(x)$,
- stores outputs in a dictionary/map,
- stops when two different inputs collide,
- prints the collision digest, number of trials, and the two colliding inputs.

Step 3: Run 20 independent experiments. For fixed t (e.g., $t = 16$), run 20 times and record the trial count at collision:

$$q_1, q_2, \ldots, q_{20}.$$

Step 4: Compare with birthday estimate. Compute the average $\bar{q}$ and compare with:

$$q \approx 1.2 \cdot 2^{t/2}.$$

6.2.1 Question (Answer Required)

After completing the collision simulation experiments and computing the average number of trials required to observe a collision, answer the following:

> **Question:** Compare your experimentally observed average collision point $\bar{q}$ with the theoretical birthday estimate $q \approx 1.2 \cdot 2^{t/2}$. How closely do your results match the theoretical prediction?
>
> In your answer:
>
> - State the value of t you used.

- Report your computed average $\bar{q}$.
- Compute the theoretical estimate for your chosen t.
- Explain why collision resistance for a full 256-bit hash would require approximately 2^{128} work.

What to Submit

- `birthday.py` script.
- A table of 20 runs with collision trial counts.
- Your computed average $\bar{q}$ and comparison to $1.2 \cdot 2^{t/2}$.
- Short explanation: why collision resistance needs large output sizes.

6.3 Task 3: Implement a Toy Merkle–Damgård Hash

Objective: Understand how iterated hashing works (compression + chaining).
Background: Many classic hashes are built by iterating a compression function over blocks:

$$V_i = h(V_{i-1}, M_i), \quad V_0 = IV, \quad H(M) = V_\ell.$$

How the Merkle–Damgård Construction Works

Modern cryptographic hash functions such as SHA-1 and SHA-256 do not process the entire message at once. Instead, they follow an **iterative (block-by-block) structure** known as the Merkle–Damgård construction.

Message Blocking. The input message M is divided into fixed-size blocks:

$$M = (M_1, M_2, \ldots, M_\ell).$$

Each block has the same length (e.g., 512 bits in SHA-256).

Initialization. Before processing begins, the algorithm starts with a fixed initial value (IV):

$$V_0 = IV.$$

This IV is defined by the hash algorithm specification.

Iterative Compression. For each block M_i, a compression function h combines:

- the previous internal state V_{i-1}, and
- the current message block M_i.

The new internal state is computed as:

$$V_i = h(V_{i-1}, M_i).$$

This process repeats sequentially for all blocks.

Final Digest. After processing all ℓ blocks, the final internal state becomes the hash output:

$$H(M) = V_\ell.$$

Why This Matters. Because each block depends on the previous internal state, every part of the message influences the final digest. Changing even one block modifies all subsequent internal states, contributing to diffusion and unpredictability. However, this same chaining structure also explains why certain constructions (such as naive prefix-based MACs) are vulnerable to length-extension attacks.

Step-by-Step Instructions

Step 1: Define a toy compression function.
In real cryptographic hash functions, the compression function is a carefully designed mathematical transformation that mixes the previous internal state with the current message block in a complex and highly nonlinear manner. For this laboratory, you will implement a simplified *toy* version to understand the structural idea without reproducing the full SHA-256 algorithm. The function

$$h(V, M_i) = \text{SHA-256}(V \,||\, M_i)$$

means that you concatenate the previous internal state V with the current message block M_i, compute the SHA-256 hash of that combined value, and then truncate the result to a smaller size (e.g., 32 or 64 bits). The truncation keeps the construction computationally manageable while preserving the chaining concept. This toy function is **not secure** and is used solely to demonstrate how iterative compression and state propagation work in the Merkle–Damgård construction.
You will implement a *toy* compression function (not secure), for learning only. Example concept:

$$h(V, M_i) = \text{SHA-256}(V \,||\, M_i) \text{ truncated to 32 bits.}$$

(You may choose 32 or 64 bits; document your choice.)
Step 2: Block parsing. Split input message into fixed-size blocks (e.g., 16 bytes). If the last block is short, pad with zeros (toy padding).
Step 3: Iterate and produce final digest.
Create `toy_md.py` that prints:

- number of blocks,
- intermediate chaining values $V_1, V_2, \ldots$ (at least for a small test message),
- final digest.

Step 4: Demonstrate chaining sensitivity. Change one byte in block M_2 and show that all subsequent V_i change.

Question (Answer Required).

In this lab, you implemented a toy Merkle–Damgård hash using a truncated compression function. Explain how the chaining values propagate changes across message blocks. Why does modifying one block affect the final digest? Additionally, discuss why truncating the output to 32 bits makes the construction insecure in practice.

What to Submit

Submit the following items for Task 3:

- The `toy_md.py` script.
- A screenshot showing the intermediate chaining values $(V_1, V_2, \ldots)$ for a small test message.
- Clear evidence demonstrating chaining sensitivity when one message block is modified.
- A one-paragraph explanation describing why this iterative structure is useful and what additional properties make real-world hash designs secure.

6.4 Task 4: Length-Extension Vulnerability

In this task, you will examine how a cryptographic hash function can be misused in practice. Although hash functions such as SHA-1 and SHA-256 are designed to provide strong security properties, improper constructions can introduce serious vulnerabilities. This lab focuses on a naive "hash-as-MAC" design, where a server attempts to authenticate messages using a simple prefix-based hash construction. Through a controlled demonstration, you will analyze why this approach is insecure and understand the structural reason behind the length-extension vulnerability.

Objective: Demonstrate and explain why the construction $\mathrm{SHA1}(k||m)$ does not provide secure message authentication.

Background: If a server computes a tag as:

$$\mathrm{tag} = \mathrm{SHA1}(\mathrm{key}||\mathrm{message}),$$

then Merkle–Damgård style hashes can be vulnerable to **length extension** where an attacker can forge a tag for:

$$\mathrm{message}||\mathrm{padding}||\mathrm{extension}.$$

Important: This is a controlled academic demonstration. You must not deploy such constructions in real systems.

Step-by-Step Instructions

Step 1: Download Your Group-Specific Artifact Package.
Each group has been assigned a unique artifact package. Download your assigned file from:

Canvas → Modules → HashServer Group XX student.zip

Download only your designated file:

`HashServer_Group_XX_student.zip`

Step 2: Unzip the Package and Inspect the Provided Values.
Unzip the file and move into the extracted folder:

```
unzip HashServer_Group_XX_student.zip
cd HashServer_Group_XX_student
```

Inspect the provided message and tag:

```
cat message.txt
cat tag.hex
cat keylen_hint.txt
```

The package contains:

- `message.txt` – the original authenticated message m,
- `tag.hex` – the original tag `tag` $= \text{SHA1}(k||m)$,
- `keylen_hint.txt` – a hint for the secret key length range.

Note: The secret key k is unknown to you and is not included in the student package.

Step 3: Define the Attacker Goal.
Your goal is to append the following data to the message:

`&role=admin`

and produce a valid tag for the extended message *without knowing the secret key.*

Step 4: Perform the Length-Extension Attack.
Using the provided `message.txt` and `tag.hex`, compute:

- a forged message: `forged_message` $= m||$`padding`$||$`&role=admin`,
- a forged tag: `forged_tag` that will verify under $\text{SHA1}(k||$`forged_message`$)$.

You must demonstrate one of the following acceptable approaches:

- **Approach A (recommended):** Use an educational length-extension tool (e.g., a standard utility available in your VM) to compute a new valid tag given the original digest and a guessed key length within the hint range.

- **Approach B (advanced):** Implement SHA-1 padding manually and explain how the internal state can continue hashing additional blocks.

Step 5: Prepare Your Submission Files.
Create and save the following outputs:

- `forged_message.txt` – the final forged message (including padding and appended data),
- `forged_tag.hex` – the forged tag in hexadecimal.

These deliverables will be verified against the instructor's hidden secret key assigned to your group. A successful verification demonstrates that authentication has been bypassed through a length-extension attack.

Step 6: Explain the Mitigation.
Explain why **HMAC** prevents this vulnerability and why:

$$\mathrm{HMAC}(k, m) \neq \mathrm{SHA1}(k||m).$$

6.4.1 Question (Answer Required)

> **Question:** Why does the construction $\mathrm{SHA1}(k||m)$ fail to provide secure message authentication, even though SHA-1 is a cryptographic hash function? In your answer, explain how the Merkle–Damgård structure enables length extension and why HMAC does not suffer from this issue.

What to Submit

For Task 4, submit the following items:

- `forged_message.txt` and `forged_tag.hex`.
- A screenshot showing the original `message` and `tag` provided in your package.
- A screenshot showing your forged message and forged tag outputs.
- A 1–2 paragraph explanation describing how the length-extension attack works and why HMAC prevents this vulnerability.

Note: Remember, do NOT submit the downloaded zip (only forged outputs).

6.5 Task 5: Password Hashing — Unsalted vs Salted vs Slow Hashing

Objective: Demonstrate why raw cryptographic hashes are insufficient for secure password storage and how salting and key stretching increase attacker cost.

Background: Hashing passwords using plain SHA-256 is vulnerable to offline dictionary and brute-force attacks. Because SHA-256 is computationally fast, attackers can test millions of guesses per second. Salting prevents precomputation attacks (e.g., rainbow tables) and ensures identical passwords do not produce identical hashes. Slow key-derivation functions (KDFs), such as PBKDF2, intentionally increase computation time per guess, significantly reducing attacker speed.

Step-by-Step Instructions

Step 1: Create a small password dictionary.
Create a file named `pwds.txt` containing 20–30 candidate passwords. Include:

- common weak passwords (e.g., `password`, `123456`),
- minor variations (e.g., `Password123`, `Winter2026!`),
- at least two longer passphrases.

Step 2: Unsalted hashing demonstration.
Write `pwd_hash.py` that:

- selects a "user password" from your dictionary,
- computes SHA-256(password),
- stores the digest in hexadecimal format,
- measures and prints the time required to compute the hash.

Step 3: Dictionary attack simulation.
Write `dict_attack.py` that:

- reads the stored digest,
- iterates through each candidate password in `pwds.txt`,
- hashes each candidate and compares it to the stored digest,
- prints:
  - recovered password (if found),
  - total guesses attempted,
  - total elapsed time,
  - estimated guesses per second.

Step 4: Add a salt.
Modify `pwd_hash.py` to generate a random salt (at least 16 bytes) and store:

$$\text{salt} \,||\, \text{SHA-256}(\text{salt}||\text{password})$$

Update `dict_attack.py` to read and use the salt during verification. Repeat the attack and compare the results.
Step 5: Implement slow hashing (PBKDF2).
Use Python's standard library to implement PBKDF2-HMAC-SHA256. Document:

- the number of iterations used,
- the time required to compute a single hash,
- the attack rate (guesses per second).

Compare the attack speed with the plain SHA-256 implementation.

6.5.1 Question (Answer Required)

After completing the experiments for unsalted hashing, salted hashing, and PBKDF2, answer the following:

> **Question:** Based on your measured attack times and guesses-per-second results, which mechanism provided the greatest increase in security: adding a salt or using PBKDF2? Justify your answer using your experimental data.
>
> In your explanation:
>
> - Explain why salts do not directly slow down guessing but still provide important security benefits.
> - Explain why fast cryptographic hashes (e.g., SHA-256) are inappropriate for password storage.
> - Describe how key stretching increases the attacker's computational cost.

What to Submit

- Scripts: `pwd_hash.py`, `dict_attack.py`.
- Screenshots demonstrating successful cracking in the unsalted case.
- Evidence showing the impact of salting and PBKDF2 on attack time.
- A short explanation (1–2 paragraphs) describing:
  - why unsalted hashes are weak,
  - why salts are necessary,
  - why slow KDFs significantly increase security.

6.6 Task 6: File Integrity Verification (Hash-Based Comparison)

Objective: Demonstrate how cryptographic hashes are used to verify file integrity and detect tampering.

Background: Cryptographic hash functions are commonly used to verify that a file has not been altered. If two files produce the same digest under a secure hash function, they are assumed to be identical. Even a single-bit change should drastically alter the hash output (avalanche effect).

Step-by-Step Instructions

Step 1: Create a binary test file.
Generate a 1 MB random binary file and create an identical copy:

```
head -c 1048576 /dev/urandom > fileA.bin
cp fileA.bin fileB.bin
```

Step 2: Compute hash digests.
Compute both SHA-1 and SHA-256 digests for the two files:

```
sha1sum fileA.bin fileB.bin
sha256sum fileA.bin fileB.bin
```

Verify that both files initially produce identical digests.
Step 3: Modify one byte in fileB.
Overwrite a single byte in `fileB.bin`:

```
printf '\x00' | dd of=fileB.bin bs=1 seek=1000 count=1 conv=notrunc
```

Step 4: Recompute and compare hashes.
Run the hash commands again and compare the outputs. Observe that changing a single byte produces a completely different digest.
Step 5: Explain the trust model.
In 1–2 paragraphs, explain:

- why a hash can detect tampering,
- why the reference hash must be obtained from a trusted source,
- what could happen if an attacker can modify both the file and the stored hash.

6.6.1 Question (Answer Required).

In this task, a one-byte modification completely changed the file's hash value. Explain why cryptographic hash functions are highly sensitive to small input changes. Additionally, discuss why comparing hashes alone does not guarantee authenticity unless the reference hash is obtained through a trusted channel.

What to Submit

- Screenshots showing the SHA-1 and SHA-256 outputs before modification.
- Screenshots showing the outputs after the 1-byte modification.
- A short explanation describing how hash-based integrity verification works and what assumptions it relies on.

6.7 Task 7: Performance Benchmarking of Hash Functions

Objective: Measure and compare the performance of multiple cryptographic hash functions and interpret throughput results.

Background: Cryptographic hash functions differ not only in security properties but also in computational performance. In real systems, performance can affect system scalability and design choices. However, speed must always be balanced against security requirements.

Step-by-Step Instructions

Step 1: Prepare datasets.
Generate random binary files of different sizes:

```
head -c 1024 /dev/urandom > data_1KB.bin
head -c 1048576 /dev/urandom > data_1MB.bin
head -c 10485760 /dev/urandom > data_10MB.bin
```

Step 2: Benchmark SHA-1, SHA-256, and SHA-512.
For each file size, run each hash algorithm **at least 5 times** and compute the average execution time.
Use:

```
time sha1sum data_1MB.bin > /dev/null
time sha256sum data_1MB.bin > /dev/null
time sha512sum data_1MB.bin > /dev/null
```

Record the `real` time reported by the system.

Step 3: Compute Throughput.
For each experiment, compute throughput using:

$$\text{Throughput (MB/s)} = \frac{\text{File Size (MB)}}{\text{Average Time (seconds)}}$$

Step 4: Report Results in a Table.
Your table must include:

- file size,
- hash algorithm,

- average execution time (seconds),
- computed throughput (MB/s).

Step 5: Analysis.
Write 1–2 paragraphs interpreting:

- why different hash algorithms have different execution speeds,
- why SHA-512 can sometimes outperform SHA-256 on 64-bit architectures,
- why performance alone should not determine cryptographic choice,
- how these results relate to password hashing (Task 5).

6.7.1 Question (Answer Required)

Based on your benchmark results, explain why different hash algorithms exhibit different performance characteristics. Why can SHA-512 sometimes be faster than SHA-256 on modern 64-bit systems? Finally, discuss why performance alone should not determine which hash function is used in a security-critical system.

What to Submit

- Benchmark evidence (screenshots or structured script output).
- A properly formatted results table.
- A short analytical discussion interpreting your findings.

Lab 4 Submission Requirements

This lab contains multiple tasks combining implementation, experimentation, and conceptual analysis. Your submission must be complete, well-organized, and reproducible.

Submission Format:
Submit a single compressed file named:

`Lab4_GroupXX.zip`

Your archive must contain:

- **A structured report (PDF)** named:

  `Lab4_Report.pdf`

  The report must:

  - clearly separate Tasks 1–7,

  - include all required tables,
  - include all requested explanations,
  - include labeled screenshots,
  - present results in a professional format.

- **All Python scripts used in the lab**, including (as applicable):
  - `avalanche.py`
  - `birthday.py`
  - `toy_md.py`
  - Task 4 attack scripts
  - `pwd_hash.py`
  - `dict_attack.py`

- **Generated data files**, including:
  - `pwds.txt`
  - forged outputs for Task 4
  - any benchmark result files (if applicable)

General Requirements:

- Code must be readable and properly commented.
- All scripts must run without modification on a standard Linux/Kali environment.
- Screenshots must clearly show command outputs (not cropped to hide errors).
- All tables must include units (seconds, MB/s, etc.).
- All experimental results must include brief interpretation.

Reproducibility Requirement:
Your submission must allow the instructor to:

- re-run your scripts,
- reproduce your experimental results,
- verify Task 4 forgery correctness,
- confirm benchmark calculations.

Failure to provide reproducible code may result in grade deductions.

7 Grading Expectations

A submission is considered **Complete** if:

- All tasks are attempted with correct methodology,
- Required evidence is included,
- Explanations show understanding (not just raw outputs),
- Results are reproducible and properly labeled.

Grading Rubric (100 Points Total)

Component	**Points**	**Evaluation Criteria**
Task 1: Avalanche Effect	10	Correct implementation, accurate bit-difference calculation, clear interpretation of results.
Task 2: Birthday Simulation	15	Correct collision script, 20 experimental runs, correct statistical comparison with the theoretical estimate.
Task 3: Merkle–Damgård Implementation	15	Correct block parsing, chaining logic, evidence of chaining sensitivity, explanation of structure.
Task 4: Length-Extension Demonstration	20	Successful attack demonstration, correct verification, clear explanation of vulnerability and HMAC mitigation.
Task 5: Password Hashing & Dictionary Attack	15	Successful attack on unsalted case, comparison with salted + PBKDF2, performance discussion.
Task 6: File Integrity Verification	10	Correct demonstration of tamper detection and explanation of trust model assumptions.
Task 7: Performance Benchmarking	10	Accurate timing measurements, structured table, throughput computation, analytical discussion.
Report Quality & Clarity	5	Organization, readability, correct labeling of screenshots and outputs.
Code Quality & Reproducibility	10	Readable scripts, correct comments, reproducibility instructions provided.
Total	**100**	